

IAgent Gateway × 4 × B100 SGLang

深度诊断报告

日期：2026-08-11 实验目标：验证“DP rank 切换 → prefix cache miss → TTFT mountain → SSE idle timeout → 空白返回/中断”的故障链。数据源：2026-08-11 gateway 黑盒实验，以及 2026-08-04 11:18:18 至 2026-08-10 05:48:21 的 272.7 MB、1,240,031 行部署日志。

1. 执行摘要

本轮数据支持一个比原假设更精确的结论：

DP=4 + round-robin + rank-local Radix cache 确实造成同一前缀在不同 GPU 上“近乎全命中”和“近乎全 miss”的强烈分化；但本轮没有观测到 60 – 90 秒 TTFT 或连接中断，且唯一 growing-context 尖峰的立即重试更慢。因此，prefix locality 是高概率放大器，但还不能单独解释“Baked 72s 后空白返回”。空白 clean-stop 与 thinking/reasoning 输出路径的关联反而非常强。

关键结果：

- 完整运行产生 132 条记录：60 条 baseline、32 条 prefix-reuse (thinking on/off 各 16)、32 条并发 session、8 条 beacon。
- 没有 Case A、B、D，也没有 client timeout、连接中断或 HTTP 错误；124 条非 beacon 记录为 83 NORMAL、41 Case C。
- 全部 41 个 Case C 都来自 thinking-on；thinking-off 的 78 条非 beacon 请求全部 NORMAL。这表明 Case C 在本轮主要是 reasoning 已产生、但没有最终 text/tool 的 clean stop，不应直接解释为“引擎完全没有生成”。
- growing-context thinking-on 在 56,518 tokens 出现一次 TTFT 4.095s：历史中位数的 7.08×，配对短控制 0.424s 的 9.67×，满足 reuse-specific mountain 定义。
- 该尖峰的立即相同 payload 重试为 10.642s，比尖峰还慢 2.60×，不满足“continue 立即恢复”；下一回合才恢复到 0.527s。
- thinking-off growing-context 的 TTFT 中位数 0.802s、最大 1.707s、0 次 spike、16/16 NORMAL。
- 并发测试中 s0/s1/s2 分别出现 14.301s、15.169s、11.719s 的孤立慢请求，而 s3 最大仅 1.757s；beacon 中位数 0.220s，最大 4.879s，全部正常。这与 rank/queue 局部慢，而非整个 engine wedge 相符。
- 246 个 late-abort 周围 ±2 秒，97.2% 存在 prefill queue；普通活跃窗口为 49.7%。queue ≥ 10 分别为 19.5% 与 2.58%，约 7.6× 富集。这把 abort 与排队压力关联起来，但仍不是单请求因果证明。
- 39 个 streaming parser 错误各自最近的 access log 全部是 gateway IP 172.16.20.131 的 OpenAI /v1/chat/completions 200，时间差 p50=2s、最大=3s；没有一条关联 Anthropic 路径。
- 历史日志没有 OOM、CUDA error、NCCL error、Traceback 或 5xx；内存占用峰值仅 full cache 27%、SWA cache 37%。因此显存耗尽或 GPU 全局崩溃不是首要方向。

机器生成的 [REPORT.md](#) 按计划中的简单计数规则给出“CONSISTENT”。该规则把“只要执行了 recovery 请求”当成 P-recovery，没有判断 recovery 是否更快，也没有把 baseline 长尾作为反证；因此应以本文的分层结论为准。

2. 实际部署拓扑：不是 vLLM，而是 SGLang

日志入口明确是 `sglang.launch_server`。核心配置集中在 [日志第 10 行](#)：

项目	实际值	诊断含义
Engine	SGLang	不能直接套用 vLLM 的 DP/cache/metrics 语义
GPU 拓扑	<code>dp_size=4, tp_size=1</code>	每张卡是独立 DP replica；KV/Radix cache 不跨卡共享
DP 路由	<code>round_robin</code>	没有 session/prefix affinity；同一长会话逐请求轮转 GPU
Rank 内调度	<code>schedule_policy=lpm</code>	只在已经选中的 rank 内做 longest-prefix-match，无法修复跨 rank miss
Context	524,288	后端窗口大于当前 gateway harness 的 131,072 假设
Chunked prefill	16,384	长 miss 被拆成许多 16k chunk
Max prefill tokens	32,768	限制单批 prefill 工作量，会拉长超长 miss 的调度轮次
Running / queued	8 / 32 per rank	burst 时存在明显排队空间
Radix cache	enabled	<code>disable_radix_cache=False</code>
Chunked prefix cache	enabled	<code>disable_chunked_prefix_cache=False</code>
Session Radix cache	disabled	<code>enable_session_radix_cache=False</code>
KV dtype	FP8 E4M3	节省 KV 显存；日志提示缺少 scaling factors，主要是精度风险
Attention	DSV4 backend	prefill CUDA graph disabled, decode CUDA graph enabled
Sampling / MoE	FlashInfer / MXFP4	DSpark speculative decode, 6 draft tokens
Watchdog	1,800s	远大于 72s；72s 中断不太可能是 SGLang watchdog
Stream interval	1	首 token 之前仍可能完全无 content 数据

DP0 – DP3 分别绑定 GPU 0 – 3，可见[日志第 17 – 20 行](#)。每个 rank 启动后拥有约 28.03 GB 可用静态池、9,216,256 total tokens；DP1 – 3 见[第 431 行](#)，DP0 见[第 501 行](#)。服务在[第 572 行](#) ready。

文件名包含 `skip_warmup`，但实际参数是 `skip_server_warmup=False`，而且日志确实执行了 FlashInfer autotune、DeepGEMM warmup 和 CUDA graph capture。这个命名差异不会解释运行期 72s 问题，但会影响启动时间判断。

SGLang 当前官方实现进一步确认：round-robin controller 只是将计数器加一后按 active rank 取模，完全不读取 queue、token load 或 prefix；同一实现还提供 `total_requests`、`total_tokens`，并允许请求携带外部 `routed_dp_rank`。参见官方 `data_parallel_controller.py`。部署日志来自定制分支，切换前仍需用该分支的

--help/源码确认可用选项。

另一个容易误读的参数是 `enable_session_radix_cache`。官方说明是：它为 session 持有的 KV 增加引用，使 eviction 优先清理未引用条目；它不是 DP 路由或跨 rank cache 共享机制。因此单独打开它不能解决 round-robin locality，只能降低当前 rank 上 active session KV 被驱逐的概率。参数说明见官方 `server_args.py`。

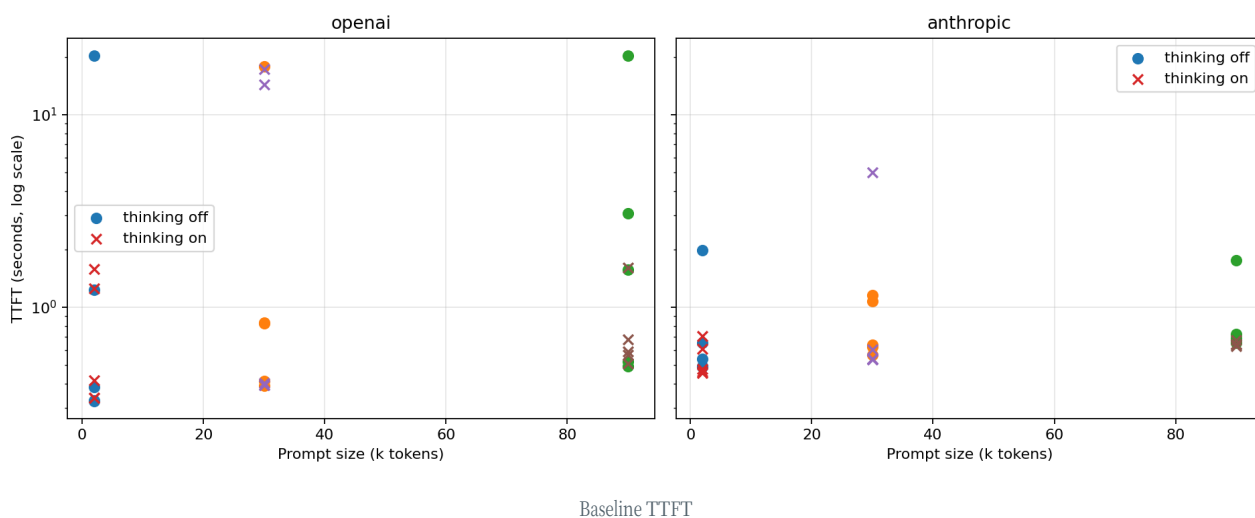
3. Gateway 黑盒实验结果

3.1 单轮 baseline

协议	样本	TTFT p50	p90	p95	最大	>3s	>10s
OpenAI	30	0.578s	17.297s	20.310s	20.342s	6	5
Anthropic	30	0.639s	1.157s	1.969s	5.008s	1	0

OpenAI 在 2k、30k、90k 三种长度都出现 17 – 20s 长尾：2k thinking-off 最大 20.310s，30k thinking-off 最大 17.868s，30k thinking-on 最大 17.297s，90k thinking-off 最大 20.342s。长尾并不随输入长度单调增长，因此不是纯 prefill 成本曲线；协议适配器、gateway 排队或 DP rank 局部负载至少参与其中。

Anthropic 路径明显更稳定。两种协议最终落到同一模型时，TTFT 尾部差异提示 gateway/adaptor 边界是必须单独排查的一层。



3.2 Growing context 与 thinking 对照

模式	回合	上下文终点	TTFT p50	最大	spike	Case C	NORMAL
thinking on	16	111,037	0.850s	4.095s	1	13	3
thinking off	16	111,037	0.802s	1.707s	0	0	16

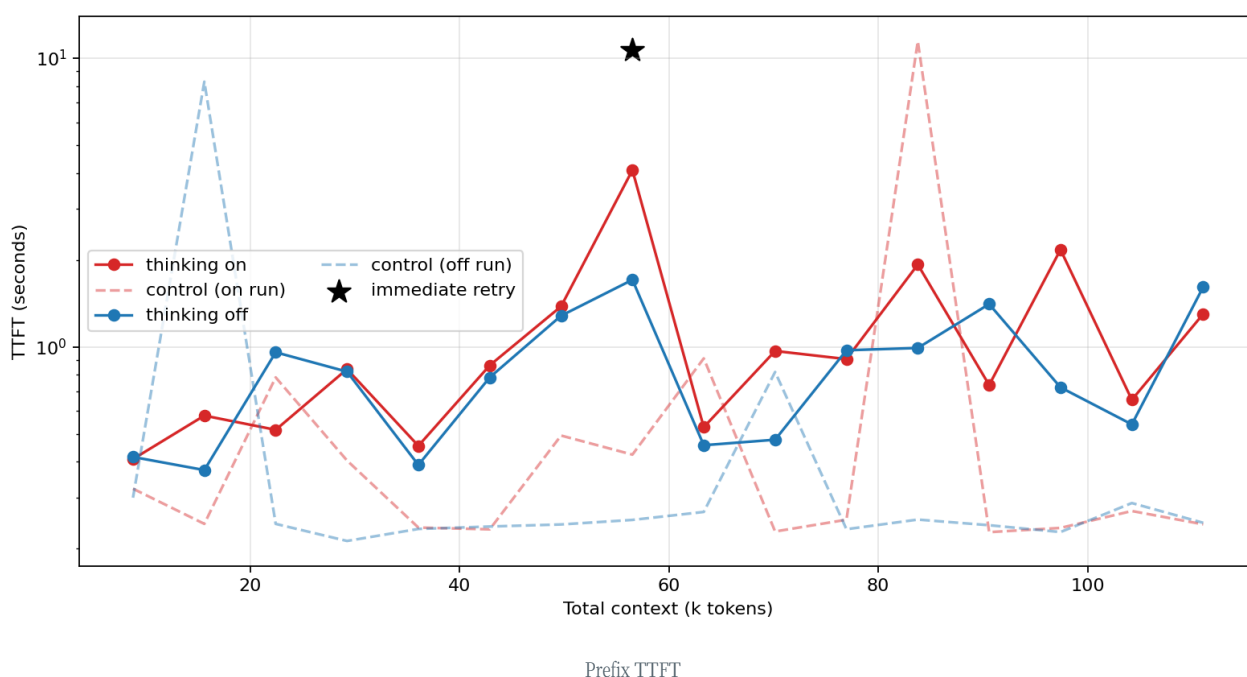
唯一 mountain 位于 turn 7、56,518 tokens：

- growing TTFT：4.095s
- 历史中位数：0.578s

- 同时短控制：0.424s
- 即时 exact-payload retry：10.642s
- 下一 growing turn：0.527s

它证明了“同一时刻长 payload 可以比短控制慢一个数量级”，但没有证明“第一次 miss、第二次 rehash 后 hit”；第二次反而更慢。结合 round-robin，合理解释是连续两个 rank 的 cache/queue 状态都不理想，而后续 rank 较快。没有 rank ID 或 request ID 时不能区分 cache miss 与 rank-local queue。

thinking-off 完全消除了 Case C，并把最大 TTFT 压在 1.707s。由于两轮不是同时随机交错运行，不能把差异全部归因于 thinking；但“41 个 Case C 全部属于 thinking-on”是非常强的适配器/输出预算信号。本 harness 的 thinking 请求仅给 128/256 tokens（尾部 64），模型可能把预算全部用于 reasoning 后以 **length/max_tokens** clean stop，客户端又隐藏 reasoning，于是用户看到“无回答”。

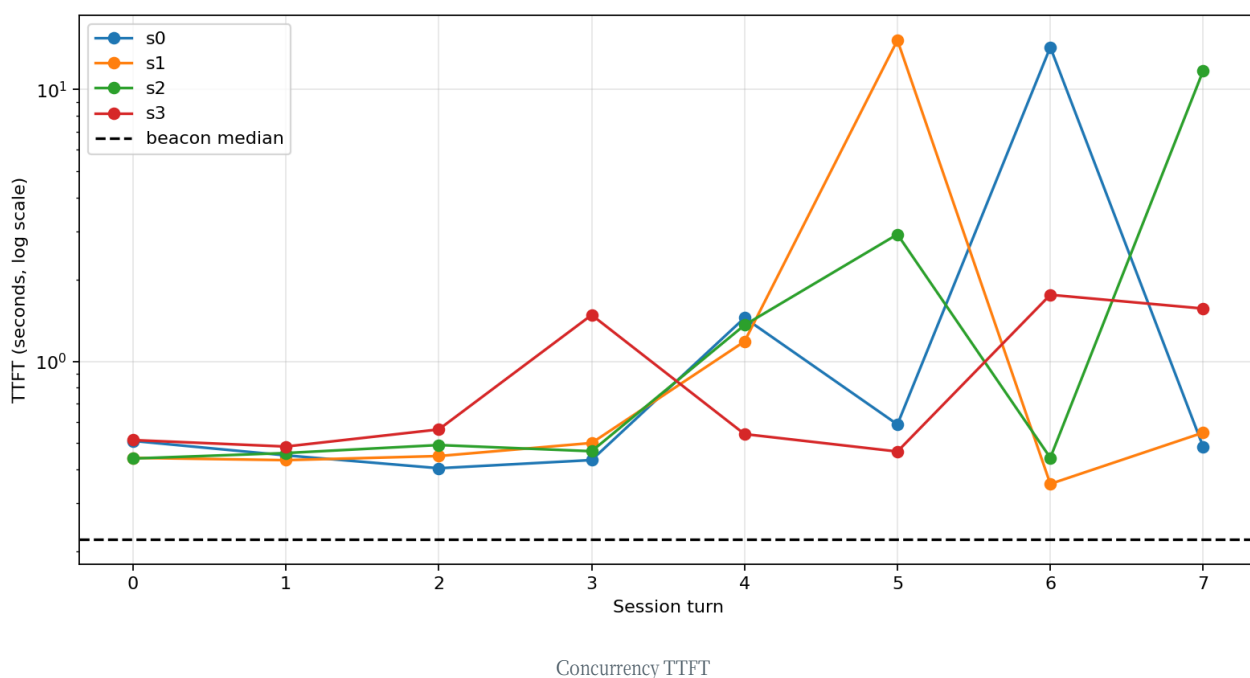


3.3 并发隔离

32 个 session turn 全部 NORMAL，但尾部高度不均衡：

Session	TTFT p50	最大
s0	0.497s	14.301s
s1	0.475s	15.169s
s2	0.480s	11.719s
s3	0.551s	1.757s
beacon	0.220s	4.879s

s0/s1/s2 的慢请求在相近尾部阶段出现，而其他会话仍可完成；随后各会话又回到约 0.35 – 0.55s。该形态更像部分 rank 同时处理长 prefix/队列，而不是四卡共同 hang。beacon 也曾升至 4.879s，说明当时存在一定共享或路由层负载，因此不能把所有 11 – 15s 都归因于 prefix miss。



4. 138.5 小时服务日志统计

4.1 请求与稳定性

指标	数量/分布
200 responses	222,038
400 responses	968 (约 0.434%)
5xx	0
Prefill batch logs	546,549
Decode batch logs	468,067
OOM / CUDA error / NCCL error / Traceback	0 / 0 / 0 / 0
Abort request ... already finished/removed	246
parse_streaming_increment errors	39 log events

Abort ... not found 表示 abort 到达时请求已经结束或被移除，不能单独视为 engine abort；它更可能是客户端/gateway 取消与服务端完成之间的竞态。日志没有 request ID 贯穿 access、prefill、decode、abort，因此无法确认这 246 条是否对应用户的空白返回。

39 条 streaming parser 错误包括非法 JSON 标点/字符，例如第 116,287 行的 **Expected ':", got e**、第 241,240 行的中文逗号；另有多组同一秒内连续 5 – 8 条错误。这更像模型生成 tool-call JSON 时的增量解析失败。它是“有生成但 adapter 无法形成最终 tool/text”的直接服务端证据，但 39 是日志事件数，不保证对应 39 个独立请求。

新增的 access-log 关联分析显示，这 39 条各自最近的 access log 全部是 **172.16.20.131** 发出的 **/v1/chat/completions** 200，时间差 p50=2s、最大=3s；Anthropic **/v1/messages** 没有对应事件。由此可把排查范围从“通用 parser”收窄到 OpenAI streaming tool/reasoning

parser。这是时间邻近关联，并非请求 ID 级映射。

400 也不是均匀来自生产 gateway：968 条 400 中，822 条来自服务本机 **192.168.89.37**（该来源 400 率 5.17%），121 条来自 **192.168.2.136**（26.13%），gateway IP **172.16.20.131** 只有 25 条（其请求 400 率 0.012%）。因此总 400 率不能用于评价 gateway 的真实失败率，多数更像本机/测试流量。

4.2 负载、排队和内存

指标	p50	p90	p95	p99	最大
Prefill queue requests	0	4	5	10	27
Pending prefill tokens（批级）	68,397	478,435	665,789	1,110,415	2,658,894
Full token usage	2%	9%	12%	17%	27%
SWA token usage	0%	2%	3%	4%	37%
Decode running requests	1	2	3	7	8
Decode throughput	335 tok/s	581	728	1,069	1,824

36.9% 的 prefill batch 日志中 queue 非零，最大 queue 27，接近每 rank 的 **max_queued_requests=32**。这说明 TTFT 长尾完全可能包含排队时间。另一方面 full/SWA cache 利用率从未接近耗尽，日志没有 retraction、OOM 或运行期 watchdog，因而“KV pool 满导致大量驱逐/重算”没有直接证据。

按每秒聚合后，DP0 – DP3 最长连续 queue-positive 区间分别为 125s、144s、148s、115s，已经覆盖并超过用户观察到的约 72s 窗口。最严重时 DP3 连续多秒 **running=7, queue=26, pending≈1.2M**，而同一秒 DP2 只有 **running=1, queue=2**，见第 1,058,431 行。这证明严格 round-robin 可以在 rank 负载高度不均时继续派发，而不是自动把新请求送到空闲 rank。

四个 rank 的 prefill 数为 136,357 / 136,898 / 135,844 / 137,450，分配极均衡，与 round-robin 一致；均衡的请求数不代表均衡的 token 工作量或 cache hit。

4.3 Rank-local cache 分化：最强的后端证据

日志存在相邻时刻的鲜明对照：

- DP1 对一个长请求连续执行 16,384-token chunk，**cached-token=0**，仍有 73,157、56,773 tokens pending，见第 1,239,944 行。
- 数秒后 DP3 的请求 **cached-token=146,944**、只需新算 512 tokens，见第 1,239,953 行。
- 紧接着 DP0 仅命中 256 tokens，需要先算 16,384 且仍 pending 161,736，见第 1,239,957 行。

这不能证明三条日志属于同一会话，但它证明了生产负载中不同 DP rank 同时存在“近乎全命中”和“近乎全 miss”的长前缀。结合 top-level round-robin 和不共享的 rank-local Radix cache，原假设的第一层在机制和现场数据上都成立。

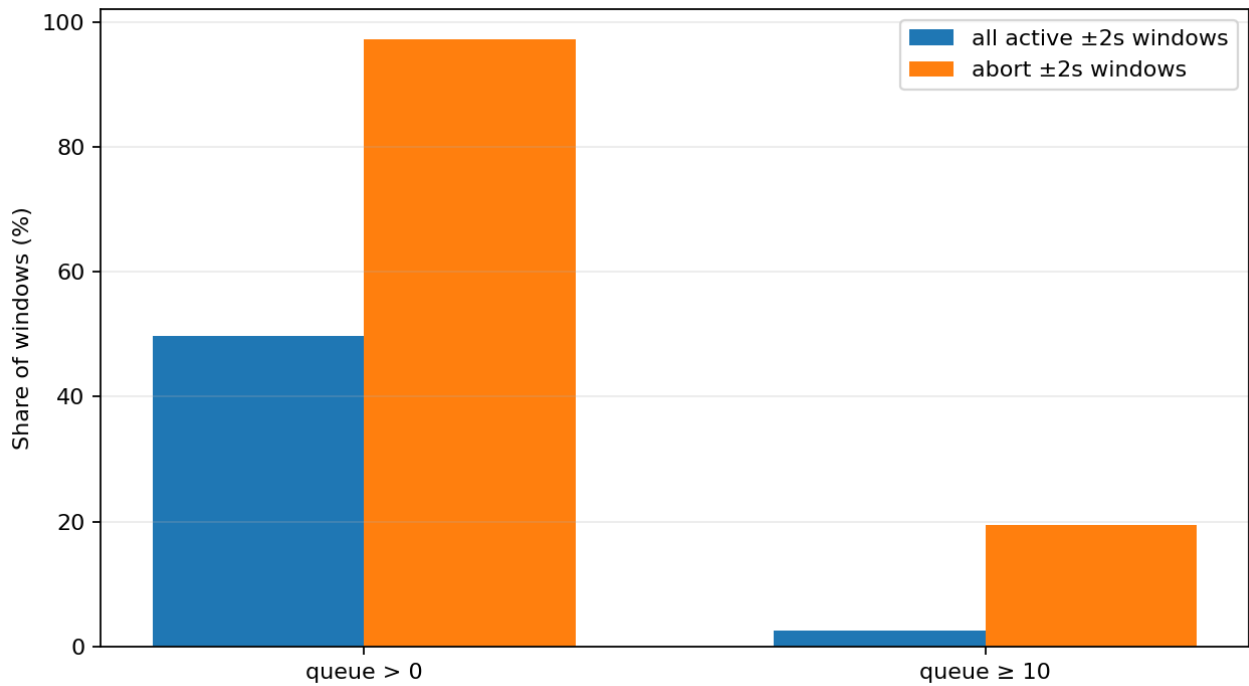
所有 prefill batch 中 32.3% 的日志 **cached-token>0**；按日志字段做批级汇总，cached token 占 **cached + new** 的约 71.3%。这只能说明 cache 整体价值很大，不能当作 request-level hit rate，因为 chunked/batched prefill 会为一个请求产生多条日志。

4.4 Abort 与 queue 的富集分析

以每个 abort 时间戳为中心取 ± 2 秒，和所有存在 prefill 日志的活跃秒做相同窗口比较：

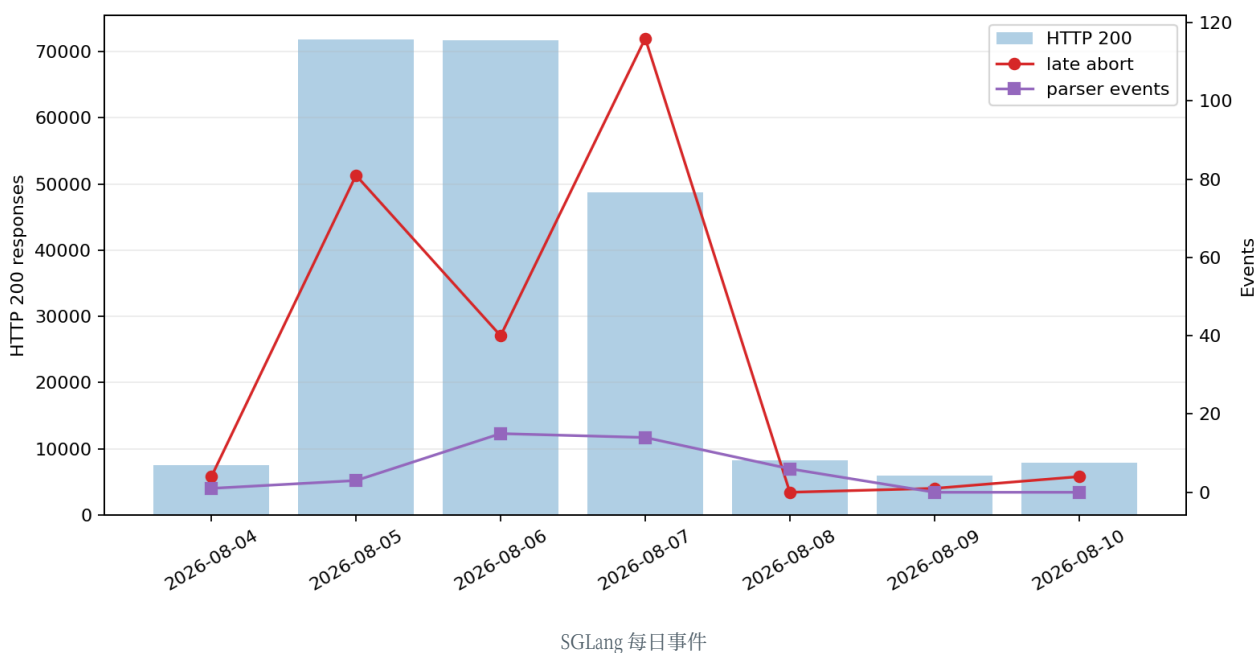
窗口	queue>0	queue $\geq$ 10	queue p50	p90	p99
所有活跃窗口	49.7%	2.58%	0	6	12
Abort 周围窗口	97.2%	19.5%	6	12	22

queue $\geq$ 10 在 abort 周围约富集 7.6 $\times$ ，queue-positive 的 odds 约富集 34 $\times$ 。这不是独立请求级因果：abort 行没有 client/rid-to-rank 映射，而且 abort 是“already finished/removed”；但它强烈说明这些竞态主要发生在负载/排队阶段，而不是均匀随机出现。



Abort 与 queue 富集

每日趋势也显示 abort 集中于高流量时段：8 月 5 – 7 日分别为 81、40、116 条，而 8 月 8 – 10 日只有 0、1、4 条。parser 错误则以簇状出现，不与 abort 一一对应。



5. 假设链逐层判定

层	判定	证据
① DP 切换 → prefix miss	强一致，但未闭环	round-robin、rank-local cache、生产日志同时出现近全 hit/miss；黑盒出现 1 次 reuse-specific mountain
② Miss → 60 – 90s TTFT	部分一致/本轮未复现	本轮最长 baseline 20.342s、并发 15.169s、prefix retry 10.642s；没有 60 – 90s
③ SSE idle timeout → abort	未观测	0 Case A/B、0 client timeout、0 conn close；SGLang watchdog 是 1800s，因此若 72s 超时存在，更可能在 gateway/CCI/client
④ Parser/adaptor empty stop	强一致	41 Case C 全部 thinking-on；服务日志另有 39 个增量解析错误事件
⑤ 全局 engine wedge	不一致	beacon 全部正常，日志无 OOM/CUDA/NCCL/5xx，内存余量充分
★ Ambient/gateway tail	强存在	OpenAI baseline 在所有 prompt size 都有 17 – 20s 长尾；短 control 也出现 11.435s

综合判定：原链条是“部分成立、缺少 timeout 闭环”，不是已经被完整证实。更符合现有数据的故障模型是：

```

round-robin 与 cache/queue 在 rank
→ TTFT 在 4-20s 范围内
→ thinking/tool JSON 与 text/tool
→ adaptor clean-stop 与 idle window
→ “Baked” 事件

```

6. 根因优先级

1. P0：DP round-robin 与 rank-local Radix cache 冲突。这是最明确的结构性缺陷。LPM 只在 rank 内有效；同一会话跨 rank 时无法利用上一回合刚建立的最长 prefix。

2. P0：thinking-only clean-stop / 输出预算 / adapter 可见性。本轮所有 Case C 都依赖 thinking-on。若 reasoning 被 UI 隐藏且 stop_reason 是 length，用户会看到“无回答”。
3. P1：rank-local prefill queue。p99 queue=10、max=27，且并发中仅部分 session 出现 11 – 15s 尾部。
4. P1：OpenAI adapter/gateway 路径额外长尾。OpenAI p90 17.3s，而 Anthropic p90 1.16s，差异不能由同一模型的 prefill 单独解释。
5. P2：tool-call incremental parser。39 个日志事件数量很小，但与 Case C 的现象方向一致。
6. 低概率：GPU/通信/显存故障。日志没有 OOM、CUDA、NCCL、Traceback、5xx 或高 cache occupancy 支持。

7. 建议与验证顺序

P0：立即做的配置 A/B

1. 停止无条件 round-robin。先 A/B 当前官方实现已有的 `total_tokens`（次选 `total_requests`），它能处理日志中“DP3 queue=26、DP2 queue=2”的负载倾斜，但仍不会优化 prefix locality。随后用 gateway session hash 或请求的外部 `routed_dp_rank` 做 sticky/prefix-aware routing。目标不是永久固定热点，而是 locality 与负载之间的受控权衡。
2. A/B `dp_size=1` 或 sticky DP=4。保持模型、负载和客户端不变，比较 TTFT p95/p99、>10s 比例和空白率。若 sticky 后 mountain 大幅消失，因果链第一层即闭环。
3. 扩大 thinking 输出预算并记录 stop reason。至少对照 64/256/1k/4k max tokens；报告 reasoning tokens、text tokens、`length/max_tokens`。UI 必须在“只有 reasoning + length”时展示明确错误，而不是静默空白。
4. 在 gateway/CCI 打印每层 timeout。明确 connect、response-header、SSE byte-idle、content-idle、total timeout；给每个请求贯穿 `gateway_request_id` → `SGLang rid` → `dp_rank`。

P1：提高可观测性

- 启用 `enable_cache_report`、`enable_request_time_stats_logging`、TTFT/inter-token latency buckets，以及 all-scheduler metrics。当前 `enable_metrics=True` 但 cache report、request timing 和 all schedulers 都关闭，无法直接做 per-rank 因果分析。官方说明 `enable_cache_report` 会把 cached token 数放进 OpenAI `usage.prompt_tokens_details`，见 `server_args.py`。
- 每个请求记录 `dp_rank`，`prompt_tokens`，`cached_tokens`，`prefill_tokens`，`queue_wait_ms`，`prefill_ms`，`first_token_ms`，`stop_reason`，`abort_source`。
- gateway 每 10 – 15s 发送 SSE comment keepalive，但客户端 TTFT 仍必须只从 content event 计算。这样可防 transport idle timeout，同时保留“engine 尚未首 token”的诊断能力。
- 对 `parse_streaming_increment` 保存脱敏后的 parser state、rid、protocol、tool name 和 stop reason；避免只留下无法关联请求的单行错误。

P2：性能与正确性

- 检查 FP8 KV scaling factors 缺失警告（第 259 行）。它更偏正确性/质量风险，不是本次 TTFT 主因。
- 评估 `chunked_prefill_size=16384` 与 `max_prefill_tokens=32768` 在 burst 下的公平性；可在 sticky routing 落地后再调整，避免同时改变两个主变量。
- 不建议先关闭 Radix cache。日志显示 cache 节省了大量 prefill；正确方向是提高 locality 和可观测性，而不是放弃 cache。

8. 下一轮最小闭环实验

建议用同一批 16 个 growing turns 做四组随机交错实验，每组至少 30 个 session：

组	DP 路由	thinking	目的
A	round-robin	on	复现当前基线
B	sticky/prefix-aware	on	隔离 DP locality
C	round-robin	off	隔离 reasoning/adaptor
D	sticky/prefix-aware	off	最低尾延迟对照

每个 turn 记录 SGLang rid/rank/cache/queue，并把 gateway content-idle timeout 临时设为大于 300s。先证明 TTFT mountain 是否随 rank/cache 改变，再恢复 60 – 90s timeout 验证是否能把 mountain 转成 Case A。这样才能把“机制一致”提升为“因果闭环”。

8.1 当前 harness 最值得补充的六项

1. 随机交错 protocol/thinking/cell 顺序。当前 baseline 按协议顺序执行，OpenAI 与 Anthropic 长尾差异可能混入时间段负载；随机交错才能做公平协议比较。
2. 输出预算 sweep。对 thinking-on 运行 `max_tokens=64/256/1024/4096`，记录 reasoning/text token、finish reason，验证 Case C 是否随预算单调下降。
3. 四连 exact-payload retry。DP=4 时不要只 retry 一次；连续四次可观察是否出现与 rank 周期一致的 TTFT/cache 模式。
4. Prefix mutation 对照。一组只在尾部 append（应命中），另一组改动前 1k tokens（强制 miss）；同一时刻配对可直接估计 cache 的 TTFT 因果效应。
5. Timeout ladder。在 gateway 侧分别设 content-idle 45/60/75/90/180s，同时保持 byte keepalive，验证 72s 是否由哪一层阈值塑造。
6. 直接后端 128k/256k/400k 阶梯。当前 gateway harness 止于 111k，无法测试历史部署 524k 窗口的真正长 prefill；需在可控后端执行，并限制并发和输出预算。

8.2 推荐统计设计

- 每个 cell 至少 30 次，预注册 p50/p95/p99、>10s、>60s、Case A/B/C 比例。
- 同时保存 matched control，并以 `(growing_ttft / control_ttft)` 为主指标，降低环境负载混杂。
- 以 session 为随机效应、rank/cache hit/queue 为解释变量；不要把同一 session 的连续 turn 当作独立样本。
- 对 retry 定义“恢复”为 `retry_ttft < min(original_ttft/2, 3×control_ttft)`，避免把“发出了 retry”误记为 recovery。

9. 局限

- 历史日志截止 2026-08-10，而黑盒实验运行于 2026-08-11；无法证明 gateway 当时一定路由到同一 SGLang 进程。
- `iagent/standard` 的 gateway 公开约束按 131,072 设计，历史后端配置为 524,288；本轮最大 111,037 tokens，没有触及后端 256k/524k 区间。
- 历史日志关闭 request logging 和 per-request timing，没有 rid-to-rank 全链路，无法把任意用户事件精确关联到 cache miss。

- SGLang throughput 日志在低负载/长日志间隔时可能包含统计窗口效应，不应把极低瞬时 throughput 直接当作单请求速度。
- 每个 baseline cell 只有 5 个样本；长尾存在性可信，但精确 p90/p95 仍需更大样本。

10. 产物

- [机器汇总报告](#)
- [Verdict JSON](#)
- [Baseline JSONL](#)
- [Prefix reuse JSONL](#)
- [Concurrency JSONL](#)
- [Probe JSON](#)
- [SGLang 日志分析 JSON](#)
- [PDF 版本](#)
- [Quick-run 归档](#)